

Category offline-first flow

This document explains how the Category feature works from application startup to local CRUD and backend synchronization. It is written for readers who are new to Flutter and should be read together with the source files linked below.

1. The main idea

The application is **offline-first**. SQLite on the device is the application's working database and the UI always reads from it.

- Creating, reading, updating, and deleting a Category does not require the internet.
- A local change is saved immediately and marked as waiting to synchronize.
- Synchronization is a separate operation. It can be started by the user or by the operating system in the background.
- If synchronization fails, the local operation is not lost.

This is the most important relationship in the application:



The UI never calls the Category API directly. This separation is what allows normal operation without an internet connection.

2. Flutter terms used in this project

Term	Meaning in this application
Widget	A piece of UI, such as the Category page or edit dialog.
Provider	A Riverpod registration that creates and supplies a dependency.
Controller	Presentation state and actions used by the UI.
Entity	The business representation of a Category.
Draft	Name, code, description, and active state entered in the form.
Repository	The domain-facing interface for Category operations.
Data source	Code that communicates with SQLite or the HTTP API.
Sync queue	Durable state stored on Category rows that records unsent work.
Tombstone	A locally deleted row retained until its server deletion succeeds.
Pull cursor	The latest server-change time already processed by this device.
Last-write-wins	The copy with the newer UTC modification time is kept.

3. Project layout

```
lib/
├─ main.dart
├─ app/
│   └─ app.dart
├─ core/
│   ├── config/app_config.dart
│   ├── database/app_database.dart
│   ├── error/app_exception.dart
│   ├── network/
│   │   ├── api_client.dart
│   │   └─ connectivity_service.dart
│   └─ sync/background_sync.dart
└─ features/category/
    ├── domain/
    │   ├── entities/category.dart
    │   ├── repositories/category_repository.dart
    │   └─ usecases/category_use_cases.dart
    ├── data/
    │   ├── datasources/
    │   │   ├── category_local_data_source.dart
    │   │   └─ category_remote_data_source.dart
    │   └─ models/
```

```

├── category_local_record.dart
├── category_remote_model.dart
├── repositories/category_repository_impl.dart
├── sync/category_sync_service.dart
└── presentation/
    ├── pages/category_list_page.dart
    ├── providers/category_providers.dart
    ├── state/category_controller.dart
    └── widgets/category_form_dialog.dart

```

Each folder has one responsibility:

- **presentation** displays data and accepts user actions.
- **domain** describes what the feature can do without depending on Flutter, SQLite, or HTTP.
- **data** implements the domain contracts.
- **core/database** owns database creation and migrations.
- **core/network** owns general HTTP and connectivity behavior.
- **core/sync** provides the operating-system background entry point.

4. Application startup

The starting point is **main.dart**.

```

main()
1. Initialize Flutter bindings.
2. Open/create the SQLite database.
3. Register the recurring background synchronization job.
4. Put the database into Riverpod's ProviderScope.
5. Render SmisApp.

```

app.dart creates **MaterialApp**, applies the theme, and opens **CategoryListPage** as the home page.

When **CategoryListPage** watches **categoryControllerProvider**, Riverpod creates **CategoryController**. Its **build()** method calls **_load()**:

```

CategoryController._load()
├── CategoryUseCases.getAll()
│   └── CategoryRepository.getAll()
│       └── CategoryLocalDataSource.getVisible()
│           └── SELECT from SQLite
├── CategoryUseCases.getPendingCount()
│   └── COUNT pending SQLite rows

```

There is no startup API request. Opening the Category page while offline works because its first read is entirely local.

5. Dependency creation with Riverpod

category_providers.dart

assembles the layers:

```

AppDatabase
└── CategoryLocalDataSource

ApiClient
└── CategoryRemoteDataSource

CategoryLocalDataSource
└── CategoryRepositoryImpl
    ├── CategoryUseCases
    └── CategoryController

Local + Remote + Connectivity
└── CategorySyncService

```

This is dependency injection: a class receives what it needs instead of creating every dependency internally. Tests can therefore replace SQLite, network connectivity, or the remote API with controlled implementations.

6. Category representations

The feature uses separate representations because the UI, SQLite, and backend do not need exactly the same fields.

- **Domain entity**

`category.dart`

contains:

- `Category` : data displayed by the application.
- `CategoryDraft` : editable form data and validation rules.
- `CategorySyncStatus` : `synced` , `pendingCreate` , `pendingUpdate` , `pendingDelete` , or `failed` .

`CategoryDraft.normalized()` trims text and enforces the same length rules as the backend: name 200, code 50, and description 500 characters.

- **Local record**

`category_local_record.dart`

contains all domain data plus offline state:

- pending operation;
- retry count and next retry time;
- deletion flag;
- last synchronization error;
- local UTC creation, update, and modification timestamps.

- **Remote model**

`category_remote_model.dart`

matches `CategoryDto` , `CategoryCreateDto` , and `CategoryUpdateDto` from the ASP.NET backend. It converts JSON responses and builds create/update payloads.

Keeping these models separate prevents SQLite-specific fields such as `retry_count` from accidentally being sent to the API.

7. SQLite database

`app_database.dart` creates the

`smis_offline.db` database. Schema version 1 has two tables.

- **categories**

Column group	Purpose
<code>id</code> , <code>name</code> , <code>code</code> , <code>description</code> , <code>is_active</code> , <code>shop_id</code>	Category data
<code>created_at</code> , <code>updated_at</code> , <code>last_modified_utc</code>	UTC audit/conflict data
<code>is_deleted</code>	Hides a local tombstone from the UI
<code>pending_operation</code>	<code>none</code> , <code>create</code> , <code>update</code> , or <code>delete</code>
<code>sync_status</code>	UI-facing synchronization status
<code>retry_count</code> , <code>next_retry_at</code> , <code>last_sync_error</code>	Durable failure/retry data

- **sync_metadata**

This key/value table stores:

- the incremental Category pull cursor;
- the synchronization lock and its owner/time.

The lock prevents a manual sync and a background sync from changing the same queue simultaneously. A lock older than ten minutes is considered stale.

Future schema changes must increment the database version and add an `onUpgrade` migration. Do not delete the user's database to make a schema change easier; that would discard offline work.

8. Read flow

When the page loads or refreshes its state:

```
CategoryListPage
→ CategoryController.reload()
→ CategoryUseCases.getAll()
→ CategoryRepositoryImpl.getAll()
→ CategoryLocalDataSource.getVisible()
→ SQLite: WHERE is_deleted = 0 ORDER BY name
→ CategoryLocalRecord.toDomain()
→ Riverpod publishes CategoryScreenState
→ Flutter rebuilds the list
```

The cloud icon on a Category card is based on `CategorySyncStatus` :

- cloud with check: synchronized;
- cloud upload: waiting to synchronize;
- cloud off: the last request failed; its tooltip shows the saved error.

9. Create flow

The user presses **Add category** and submits `CategoryFormDialog` .

```
CategoryListPage._create()
  → CategoryController.create(draft)
  → CategoryUseCases.create(draft)
  → CategoryRepositoryImpl.create(draft)
    1. Validate and trim the draft.
    2. Generate a UUID on the device.
    3. Set UTC timestamps.
    4. Set pending_operation = create.
    5. Set sync_status = pendingCreate.
  → CategoryLocalDataSource.put(record)
  → SQLite INSERT
  → Controller reloads local data
  → New Category appears immediately
```

No API call occurs in this flow. The backend supports client-provided IDs, so the same UUID can later be sent in `POST /api/Category` .

10. Update flow

The user selects **Edit**, changes the form, and saves.

```
CategoryListPage._edit()
  → CategoryController.updateCategory(id, draft)
  → CategoryUseCases.update(id, draft)
  → CategoryRepositoryImpl.update(id, draft)
    1. Read the existing local row.
    2. Validate and normalize the draft.
    3. Generate a UTC modification time newer than the previous one.
    4. Preserve operation=create if it has never reached the server.
       Otherwise set operation=update.
    5. Clear previous retry/error state.
  → SQLite UPDATE/REPLACE
  → Controller reloads local data
```

Preserving `create` is important. If a newly created Category is edited three times before its first sync, the server only needs one final create request, not one create followed by three updates.

11. Delete flow

Deletion behaves differently depending on whether the server knows the row.

- **Never synchronized**

If `pending_operation` is `create`, the record only exists on this device. Deleting it physically removes the local row. There is nothing to delete on the server and no queue item remains.

- **Previously synchronized**

The repository sets:

- `is_deleted = 1` so the UI hides it immediately;
- `pending_operation = delete` ;
- `sync_status = pendingDelete` ;
- a new UTC modification timestamp.

The row remains as a tombstone until `DELETE /api/Category/{id}` succeeds or the server reports that the row no longer exists. This makes deletion durable over app restarts and connection loss.

12. Manual synchronization

The user presses the Sync icon in the page's app bar.

```
CategoryListPage._sync()  
  → CategoryController.syncNow()  
  → CategorySyncService.synchronize(force: true)
```

`force: true` means a manual request retries all pending rows immediately, including rows whose automatic backoff time has not arrived.

The sync service performs these phases.

- **Phase A: acquire the lock**

It stores a unique owner UUID and UTC lock time in `sync_metadata` . If another sync owns a fresh lock, this run exits without starting a second copy.

- **Phase B: connectivity hint**

`connectivity_plus` checks whether some network transport exists. This is only a quick hint; Wi-Fi does not guarantee internet access. Dio still handles actual timeouts, connection failures, and HTTP errors.

- **Phase C: pull server changes**

```
Read category_pull_cursor from SQLite
→ GET /api/Category/pull?changedSince={cursor}
→ Merge every returned Category or deletion
→ Advance cursor after a successful pull
```

The cursor retains a one-millisecond overlap because the backend query uses a strict `LastModifiedUtc > changedSince` comparison. Reprocessing a boundary row is safe and helps avoid missing equal-timestamp changes.

- **Phase D: push local changes**

Eligible pending records are processed oldest first. Before changing the server, the service calls `GET /api/Category/{id}` so it can compare the latest server timestamp again.

Local operation	Server state	Action
Create/update	Missing	<code>POST</code> with the client UUID and local timestamps
Create/update	Older than local	<code>PUT</code> the local version
Create/update	Newer/equal	Apply server version locally
Delete	Missing	Remove the local tombstone
Delete	Older than local	Send <code>DELETE</code> , then remove tombstone
Delete	Newer/equal	Restore/apply the server version locally

- **Phase E: publish the result**

The controller reloads SQLite and displays pulled, pushed, conflict, failure, and remaining-pending counts. A failed network operation leaves the row queued.

13. Last-write-wins conflict handling

Every local edit receives a UTC `lastModifiedUtc`. The backend DTOs accept and preserve this value. When two copies differ:

```
local timestamp > server timestamp → local wins and is pushed
server timestamp > local timestamp → server wins and is applied locally
server timestamp = local timestamp → server wins the deterministic tie
```

Server-wins-on-tie ensures different devices make the same decision. The rule applies to normal updates and deletion conflicts.

Because timestamps decide the winner, device clocks should be reasonably synchronized. A future server-issued logical version would be more robust in environments where device clocks cannot be trusted.

14. Failure and retry behavior

The API client separates failures into two categories:

Type	Examples	Behavior
Transient	timeout, connection loss, HTTP 408/429/5xx	Save failure and retry later; stop the current push loop to avoid repeated failing calls.
Permanent	validation, authentication, most other 4xx responses	Save the error and continue with other records when possible.

Failed rows store the error, retry count, and next eligible time. Automatic delays are 1, 2, 4, 8, 16, and 32 minutes. Background selection stops picking a row after five automatic attempts, but the manual Sync button can always force another attempt.

Local CRUD is not rolled back because synchronization failed. The device copy remains useful and the pending badge remains visible.

15. Background synchronization

`background_sync.dart` contains the top-level Workmanager callback. The operating system runs this callback in a separate Dart isolate.

```
Operating system starts scheduled task
→ callbackDispatcher()
→ initialize Flutter/plugin registration in background isolate
→ open the same SQLite database
→ recreate local, remote, connectivity, and sync services
→ CategorySyncService.synchronize(force: false)
→ close the background database handle
```

`force: false` respects `next_retry_at` and the automatic retry limit.

The job is registered hourly with a connected-network constraint. This is a request to Android/iOS, not an exact timer. Battery policy, device usage, and OS limits decide the real execution time. iOS setup is present in `Info.plist` and `AppDelegate.swift`; Android uses WorkManager's automatic setup.

16. Backend API contract

The implementation was derived from `CategoryController.cs` and the DTOs under `SMIS.Application/DTO/Categories`.

Method	Endpoint	Flutter use
POST	<code>/api/Category</code>	Push a local create
GET	<code>/api/Category</code>	Available but not used for incremental sync
GET	<code>/api/Category/{id}</code>	Read current server timestamp before push
PUT	<code>/api/Category/{id}</code>	Push a newer local update
DELETE	<code>/api/Category/{id}</code>	Push a local tombstone
GET	<code>/api/Category/pull? changedSince= ...</code>	Pull incremental updates/deletions

The backend derives `ShopId` from the authenticated user. Flutter does not let the Category form choose or send a shop ID.

The deliberate test exception that previously stopped Category creation was removed from `CategoryCreateCommand` so offline creates can reach the server.

- **Current backend edge case**

The server uses soft deletion and its normal `GET` hides deleted rows. If a newer offline client edit is intended to resurrect a server-deleted Category, the current API has no explicit restore/upsert operation. Such a request remains failed and queued instead of silently discarding the local data. Supporting resurrection fully requires a backend restore endpoint or update command that can include deleted rows.

17. Configuration

`_app_config.dart` reads build definitions:

- `SMIS_API_BASE_URL` overrides the API address;
- `SMIS_AUTH_TOKEN` supplies a development token when required.

Android emulator defaults to `http://10.0.2.2:5238`. Other supported native platforms default to `http://127.0.0.1:5238`.

```
flutter run `
  --dart-define=SMIS_API_BASE_URL=https://api.example.com `
  --dart-define=SMIS_AUTH_TOKEN=development-token
```

Do not compile a long-lived production token into the application. When an authentication feature is added, it should provide short-lived tokens from secure device storage. Category CRUD will remain local and independent of that online session.

18. Tests

The current tests cover:

- create, edit, and delete without a remote API;
- physical deletion of a never-synced Category;
- tombstone retention for a server-backed Category;
- a newer local write winning and being pushed;
- a newer server write winning and replacing local pending data;

- transient failure retry metadata;
- rendering the empty Category screen.

Run them with:

```
flutter analyze
flutter test
```

19. Following one Category through its lifetime

1. User creates “Beverages” offline.
SQLite: pending create, visible immediately.
2. User renames it to “Drinks” before syncing.
SQLite: still one pending create, now containing the final name.
3. Background sync has no connection.
SQLite: unchanged; no data is lost.
4. User presses Sync after reconnecting.
Pull runs, server does not have the UUID, POST creates “Drinks”.
SQLite: operation none, status synced, ShopId filled from server response.
5. Another device updates it later.
Next pull sees the newer server timestamp and applies that version locally.
6. User deletes it while offline.
SQLite: tombstone retained but hidden from the list.
7. A later sync sends DELETE successfully.
SQLite: tombstone is physically removed.

20. Adding another entity

Do not copy files mechanically before inspecting that entity's backend contract. IDs, tenant fields, DTOs, deletion behavior, pull endpoints, and validation can differ.

The reusable Codex instructions are in

[_skills/smis-flutter-offline-entity/SKILL.md_](#).

Ask Codex to use `$smis-flutter-offline-entity` and name exactly one entity to implement.